BITWISE OPERATORS:

Bitwise operators are special operators in Python that perform operations directly on the binary (bit-level) representation of integers. They manipulate individual bits (0s and 1s) rather than entire numbers, making computations faster and more efficient.

OPERATOR	DESCRIPTION	SYNTAX
&	Bitwise AND	x & y
|	Bitwise OR	x | y
~	Bitwise NOT	~x
^	Bitwise XOR	x ^ y
>>	Bitwise right shift	x>>
<<	Bitwise left shift	x<<

These operators are commonly used in system programming, networking, cryptography, embedded system performance optimization.

1. Bitwise AND (&) :

The Bitwise AND operator compares the corresponding bits of two nu and returns 1 only if both bits are 1. Otherwise, it returns 0.

USE: Used for checking whether specific bits are set and in permission handling.

Example:

In [4]:
```
#Bitwise AND
a = 5
b = 3
print(a & b)
```

```
1
```

1. Bitwise OR (|) :

The Bitwise OR operator compares corresponding bits and returns 1 least one of the bits is 1.

USE: Used for combining flags and setting specific bits.

Example:

In [5]:
```
#Bitwise OR
a = 4
b = 6
print(a | b)
```

```
6
```

1. Bitwise XOR (^) :

The Bitwise XOR (Exclusive OR) operator returns 1 only when the corresponding bits are different.

USE: Used in encryption, error detection, and swapping two numbers without using a temporary vari

Example:

In [6]:
```
#Bitwise XOR
a = 6
b = 8
print(a ^ b)
```

```
14
```

1. Bitwise NOT (~) :

The Bitwise NOT operator inverts all the bits of a number. Every 1 becomes 0 and every 0 becomes 1.

USE: Used to invert bits and perform bit-level calculations.

Example:

In [9]:
```
#Bitwise NOT
a = 7
print(~a)
```

```
-8
```

1. Bitwise left shift (<<) :

The Left Shift operator shifts all bits to the left by a specified number of positions. Each left shift generally multiplies the numbe 2.

USE: Used for fast multiplication and binary data manipulation.

Example:

In [10]:
```
#Bitwise left shift
a = 6
print(a << 2)
```

```
24
```

1. Bitwise right shift (>>) :

The Right Shift operator shifts all bits to the right by a specifi number of positions. Each right shift generally divides the number (discarding any remainder).

USE: Used for fast division, memory management, and bit-level optimization.

Example:

In [11]:
```

```

```python
#Bitwise right shift
a = 20
print(a >> 2)
```

```
5
```

By the conclusion:

```
        Each bitwise operator performs a specific operation on the
binary representation of integers. AND (&) finds common set bits, O
(|) combines bits, XOR (^) identifies different bits, NOT (~) inver
bits, Left Shift (<<) multiplies by powers of 2, and Right Shift (>
divides by powers of 2.
```